INTELLIGENT NOSQL DATABASES:
PROJECT REPORT

MONGODB-BASED EXPENSE TRACKER SYSTEM

Submitted by

Tenzin Choeyang Dhargyal

Registration No – 12310784

Programme and Section – B.Tech. & K3P23GR

Course Code – CSE 494

Under the Guidance of

Sir Ansh Pratap Singh & U.ID – 35130

Discipline of CSE/IT

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara





CERTIFICATE

This is to certify that Tenzin Choeyang Dhargyal bearing Registration no. 12310784 has completed CSE494 project titled, “MONGODB-BASED EXPENSE TRACKER SYSTEM” under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Sir Ansh Pratap Singh

Assistant Professor

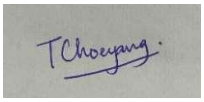
School of computer Science and Engineering

Lovely Professional University Phagwara,
Punjab.

Date: 19-10-2026

DECLARATION

I, Tenzin Choeyang Dhargyal, student of B.Tech. under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.



Date: 19-10-2026

Registration No. 12310784 Signature

Acknowledgement

I would like to express my sincere gratitude to all those who have supported and guided me throughout the completion of this project.

First and foremost, I would like to thank Sir Ansh Pratap Singh for their valuable guidance, encouragement, and constant support during the course of this project. Their expertise and insightful suggestions greatly contributed to the success of this work.

Lastly, I extend my heartfelt thanks to my family and friends for their motivation, patience, and understanding throughout this journey.

Tenzin Choeyang Dhargyal
K3P23GR
12310784

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Tools and Technologies
5. Source of Dataset
6. Data Preprocessing
7. System Design
8. MongoDB Validation
9. CRUD Operations Using PyMongo
10. Aggregation Queries and Analysis
11. AI-Assisted Receipt Entry
12. Python–MongoDB Integration
13. Results and Discussion
14. Conclusion
15. Future Scope
16. References

INTRODUCTION

Managing personal expenses is important for financial planning and budgeting. However, recording daily expenses manually makes it difficult to identify spending patterns, compare monthly expenditures, and determine which categories consume the most money.

The Expense Tracker System is a MongoDB-based application developed using Python and PyMongo. It stores transaction records as documents in a MongoDB collection and supports Create, Read, Update and Delete (CRUD) operations. MongoDB $jsonSchema validation rules ensure that stored records follow the required structure and data constraints.

The application also uses aggregation pipelines to generate meaningful information, including category-wise expenditure, monthly income versus expense, spending by payment mode and per-category statistics. The results are visualised using Matplotlib. In addition, the system can read a photographed receipt using a Gemini model and convert it into a structured transaction document that is inserted into the database.

PROBLEM STATEMENT

The objective of this project is to design and implement a database application that helps users record, manage and analyse personal income and expenses.

The system aims to:

- Store transaction information in a structured document format.
- Validate transaction records before they are stored.
- Perform CRUD operations using PyMongo.
- Retrieve useful financial insights through MongoDB aggregation pipelines.
- Reduce manual data entry by extracting transaction details from receipt images.
- Demonstrate Python integration with MongoDB.

OBJECTIVES

1. Design a suitable MongoDB database and document structure.
2. Clean and prepare a real-world transaction dataset using Pandas.
3. Implement collection-level validation rules using JSON Schema.
4. Perform insertion, retrieval, updating and deletion of transaction records.
5. Use aggregation pipelines to analyse spending patterns.
6. Visualise aggregation results with Matplotlib.
7. Connect Python with MongoDB using PyMongo.
8. Display and interpret query results.

TOOLS AND TECHNOLOGIES

Technology	**Purpose**
Python	Application logic
MongoDB	NoSQL document database
PyMongo	Python–MongoDB integration
Pandas	Data loading, cleaning and transformation
Matplotlib	Visualisation of aggregation results
KaggleHub	Downloading the source dataset
Google Gemini API (google-genai)	Extracting transaction details from receipt images
ipywidgets and Pillow	Receipt image upload and handling in the notebook
python-dotenv	Loading the API key from an environment file
Jupyter Notebook	Implementation and demonstration
MongoDB Compass (optional)	Visual inspection of databases and documents

SOURCE OF DATASET

The project uses the **Daily Household Transactions** dataset published on Kaggle by *prasad22* (prasad22/daily-transactions-dataset). It was downloaded programmatically using the kagglehub library. The raw file, *Daily Household Transactions.csv*, contains **2,461 records** with eight columns:

Column	**Description**
Date	Date and time of the transaction
Mode	Payment mode (Cash, Credit Card, Saving Bank account 1, etc.)
Category	Main category of the transaction
Subcategory	More specific classification
Note	Free-text description
Amount	Transaction amount
Income/Expense	Whether the record is an income or an expense
Currency	Currency of the amount (INR)

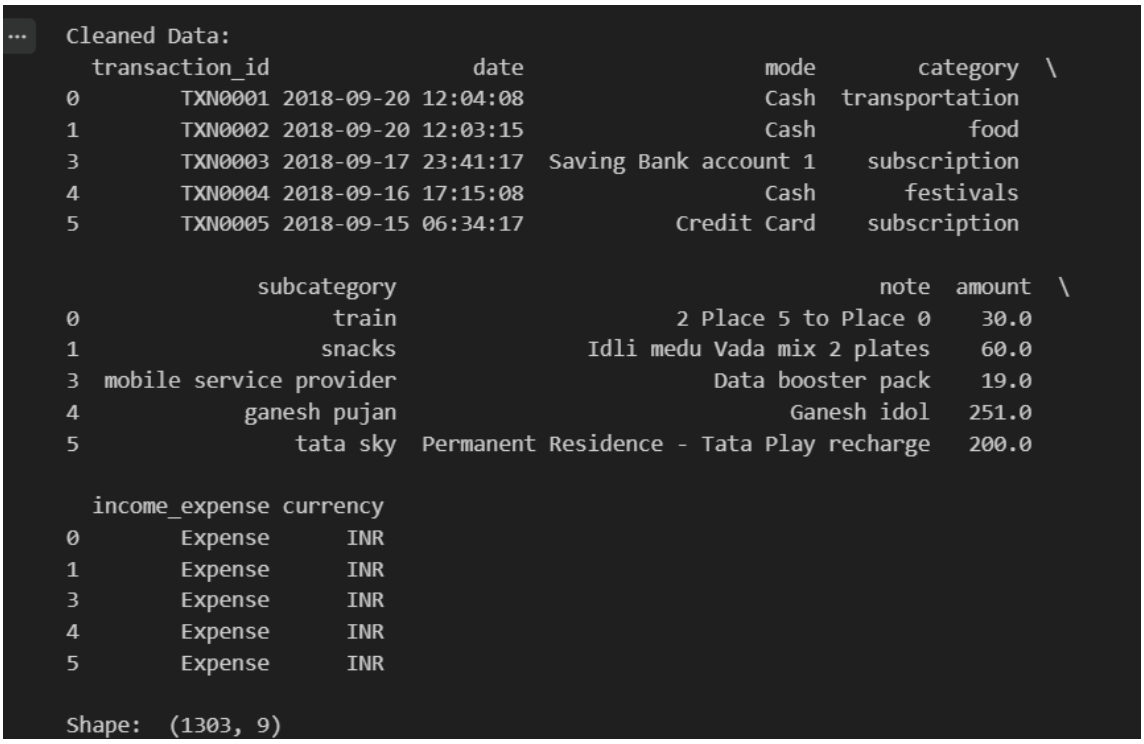
Data link: https://www.kaggle.com/datasets/prasad22/daily-transactions-dataset

DATA PREPROCESSING

Before loading data into MongoDB, the raw CSV was cleaned in Pandas so that it matches the structure enforced by the collection validator.

1. **Duplicate removal:** drop_duplicates() removed repeated rows.
2. **Column normalisation:** Column names were stripped of spaces, converted to lower case, and / and spaces were replaced with underscores (for example, Income/Expense became income_expense).
3. **Type conversion:** date was converted to a datetime type using pd.to_datetime() and amount to a numeric type using pd.to_numeric(). Invalid values were coerced to missing values.
4. **Text standardisation:** category and subcategory were stripped and converted to lower case so that entries such as "Food" and "food" are grouped together during aggregation.
5. **Missing-value handling:** Rows with a missing date, amount or category were dropped, and remaining missing values were converted to None so that they are stored as null in MongoDB.
6. **Unique identifier:** A transaction_id column (TXN0001, TXN0002, …) was generated for every record.
7. **Export:** The cleaned data was saved as transactions_cleaned.csv in the data/processed folder.

After preprocessing, the dataset shrank from **2,461 rows to 1,303 rows** and from 8 to 9 columns (with the new transaction_id).



```
Cleaned Data:
  transaction_id                date                    mode        category  \
0        TXN0001 2018-09-20 12:04:08                    Cash  transportation
1        TXN0002 2018-09-20 12:03:15                    Cash            food
3        TXN0003 2018-09-17 23:41:17  Saving Bank account 1    subscription
4        TXN0004 2018-09-16 17:15:08                    Cash       festivals
5        TXN0005 2018-09-15 06:34:17             Credit Card    subscription

                subcategory                                        note  amount  \
0                     train                        2 Place 5 to Place 0    30.0
1                    snacks                 Idli medu Vada mix 2 plates    60.0
3  mobile service provider                             Data booster pack    19.0
4              ganesh pujan                                  Ganesh idol   251.0
5                  tata sky  Permanent Residence - Tata Play recharge   200.0

  income_expense currency
0        Expense      INR
1        Expense      INR
3        Expense      INR
4        Expense      INR
5        Expense      INR

Shape:  (1303, 9)
```

SYSTEM DESIGN

- **Database Design**

  The application uses a database named expense_db and a collection named transactions. Each transaction is stored as an independent MongoDB document, so all fields describing one transaction are kept together in a single record.

- **Document Structure**

  Example document, as stored in the collection:

```
{
 "_id": "ObjectId generated by MongoDB",
 "transaction_id": "TXN0002",
 "date": "2018-09-20T12:03:15",
 "mode": "Cash",
 "category": "food",
 "subcategory": "snacks",
 "note": "Idli medu Vada mix 2 plates",
 "amount": 60.0,
 "income_expense": "Expense",
 "currency": "INR"
}
```

  The _id field is generated automatically by MongoDB. The date field is stored as a native BSON date (Python datetime), which makes date-based aggregation using $year and $month possible.

- **Field Description**

Field	Data Type	Description
_id	ObjectId	Unique document identifier
transaction_id	String	Application-level transaction code
date	Date	Date and time of the transaction
mode	String	Payment mode
category	String	Transaction category
subcategory	String / null	More specific classification
note	String / null	Short description
amount	Number	Transaction amount

income_expense	String	"Income" or "Expense"
currency	String	Currency code (INR)

MONGODB VALIDATION

Collection validation is implemented with the $jsonSchema validator when the transactions collection is created using create_collection(). The notebook creates the collection only if it does not already exist.

The validation rules ensure that:

- date, mode, category, amount, income_expense and currency are required fields.
- date is a BSON date.
- mode and category are strings.
- subcategory and note are strings or null.
- amount is a double, int or long, with a minimum value of 0.
- income_expense must be either "Income" or "Expense".
- currency must be "INR".

These rules improve data consistency and prevent invalid documents from being inserted into the collection.

```
db=client["expense_db"]
if "transactions" not in db.list_collection_names():
    db.create_collection(
        "transactions",
        validator={
            "$jsonSchema": {
                "bsonType": "object",
                "required": [
                    "date",
                    "mode",
                    "category",
                    "amount",
                    "income_expense",
                    "currency"
                ],
                "properties": {
                    "date": {
                        "bsonType": "date"
                    },
                    "mode": {
                        "bsonType": "string"
                    },
                    "category": {
                        "bsonType": "string"
                    },
                    "subcategory": {
                        "bsonType": ["string", "null"]
                    },
                    "note": {
                        "bsonType": ["string", "null"]
                    },
                    "amount": {
                        "bsonType": ["double", "int", "long"],
                        "minimum": 0
                    },
                    "income_expense": {
                        "enum": ["Income", "Expense"]
                    },
                    "currency": {
                        "enum": ["INR"]
                    }
                }
            }
        }
    )
collection=db["transactions"]
```

CRUD OPERATIONS USING PYMONGO

1. Create Operation

The cleaned records were converted to a list of dictionaries with df.to_dict(orient="records") and loaded using insert_many(). A single manually entered transaction was added using insert_one():



```python
expense = {
    "transaction_id": "TXN0008",
    "date": datetime(2026, 9, 15),
    "mode": "UPI",
    "category": "Food",
    "subcategory": "Pizza",
    "note": "Dominos",
    "amount": 450.0,
    "income_expense": "Expense",
    "currency": "INR"
}

collection.insert_one(expense)
```

Output: InsertOneResult(ObjectId('6ab0c9a1f56bd860021fdd4c'), acknowledged=True). MongoDB accepted the document because it satisfies all validation rules.



```python
expense = {
    "transaction_id": "TXN0008",
    "date": datetime(2026, 9, 15),
    "mode": "UPI",
    "category": "Food",
    "subcategory": "Pizza",
    "note": "Dominos",
    "amount": 450.0,
    "income_expense": "Expense",
    "currency": "INR"
}

collection.insert_one(expense)
```

Python

```
InsertOneResult(ObjectId('6ab0c9a1f56bd860021fdd4c'), acknowledged=True)
```

2. Read Operation

Stored records were retrieved using find(). Three read queries were demonstrated:

- All documents: collection.find()
- Combined filter with $and (category "food" and subcategory "snacks"). The query returned snack purchases such as TXN0002 (idli medu vada, ₹60), TXN0047 (₹32), TXN0082 (₹60) and TXN0094 (₹84).
- Comparison filter: {"amount": {"$gt": 500}}, which returned high-value transactions such as TXN0010 (family, ₹1,000), TXN0012 (food, ₹510.85) and TXN0028 (food, ₹1,120).

Purpose: View all transactions and retrieve specific records.

```python
for transaction in collection.find():
    print(transaction)
```

Python

```
ObjectId('6aaa76b0b9f553990a97e516'), 'transaction_id': 'TXN0001', 'date': datetime.datetime(2018, 9, 20, 12, 4, 8),
ObjectId('6aaa76b0b9f553990a97e517'), 'transaction_id': 'TXN0002', 'date': datetime.datetime(2018, 9, 20, 12, 3, 15)
ObjectId('6aaa76b0b9f553990a97e518'), 'transaction_id': 'TXN0003', 'date': datetime.datetime(2018, 9, 17, 23, 41, 17
ObjectId('6aaa76b0b9f553990a97e519'), 'transaction_id': 'TXN0004', 'date': datetime.datetime(2018, 9, 16, 17, 15, 8)
ObjectId('6aaa76b0b9f553990a97e51a'), 'transaction_id': 'TXN0005', 'date': datetime.datetime(2018, 9, 15, 6, 34, 17)
ObjectId('6aaa76b0b9f553990a97e51b'), 'transaction_id': 'TXN0006', 'date': datetime.datetime(2018, 9, 14, 5, 39, 17)
ObjectId('6aaa76b0b9f553990a97e51c'), 'transaction_id': 'TXN0007', 'date': datetime.datetime(2018, 9, 13, 21, 35, 15
```

3. Update Operation

The update operation was performed with `update_one()` and the `$set` operator:

```python
result=collection.update_one({"subcategory":"curd"},{"$set": {"amount":40}})
print("Modified:", result.modified_count)
```

Output: Modified: 0. No document matched the filter subcategory = "curd", so nothing was changed. This confirms that update_one() modifies a document only when the filter matches an existing record.

```python
result=collection.update_one({"subcategory":"curd"},{"$set": {"amount":40}})
print("Modified:", result.modified_count)
```

```
Modified: 0
```

4. Delete Operation

A transaction was removed using `delete_one()` with a combined filter so that only the intended record is affected:

```python
result=collection.delete_one({"$and":[{"amount":{"$gt":5000}},{"category":"salary"}]})
print("Deleted:",result.deleted_count)
```

Output: Deleted: 1. One salary transaction above ₹5,000 was removed.

```python
result=collection.delete_one({"$and":[{"amount":{"$gt":5000}},{"category":"salary"}]})
print("Deleted:",result.deleted_count)
```

```
Deleted: 1
```

AGGREGATION QUERIES AND ANALYSIS

MongoDB aggregation pipelines process documents through stages such as $match, $group, $sort, $project and $limit to produce summarised results.

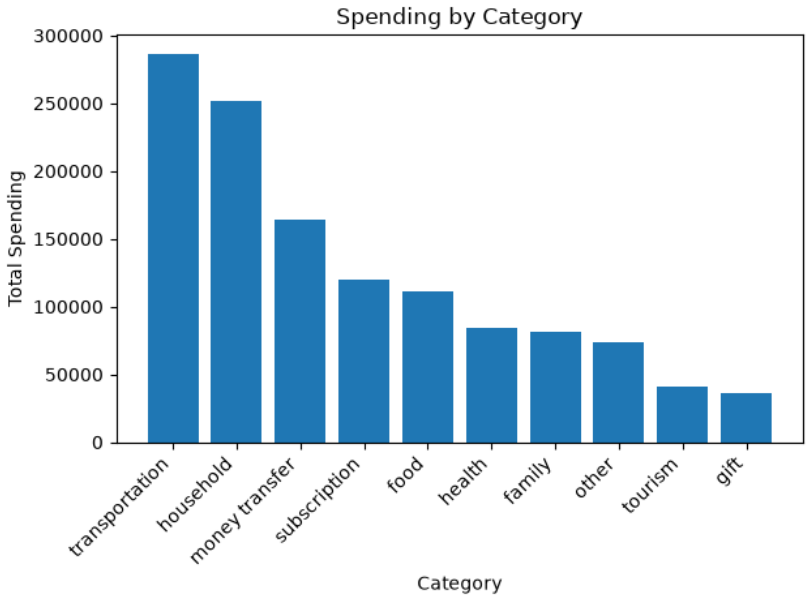
1. Category-wise Spending

The pipeline filters expense records, groups them by category, sums the amounts, sorts in descending order and returns the top ten categories.

```
total_spending_by_category=[{"$match":{"income_expense":"Expense"}},
                            {"$group":{"_id":"$category","totalSpend":{"$sum":"$amount"}}},
                            {"$sort":{"totalSpend":-1}},
                            {"$project":{"_id":0,"category":"$_id","totalSpend":1}},
                            {"$limit":10}]

result1=list(collection.aggregate(total_spending_by_category))

print(result1)
```

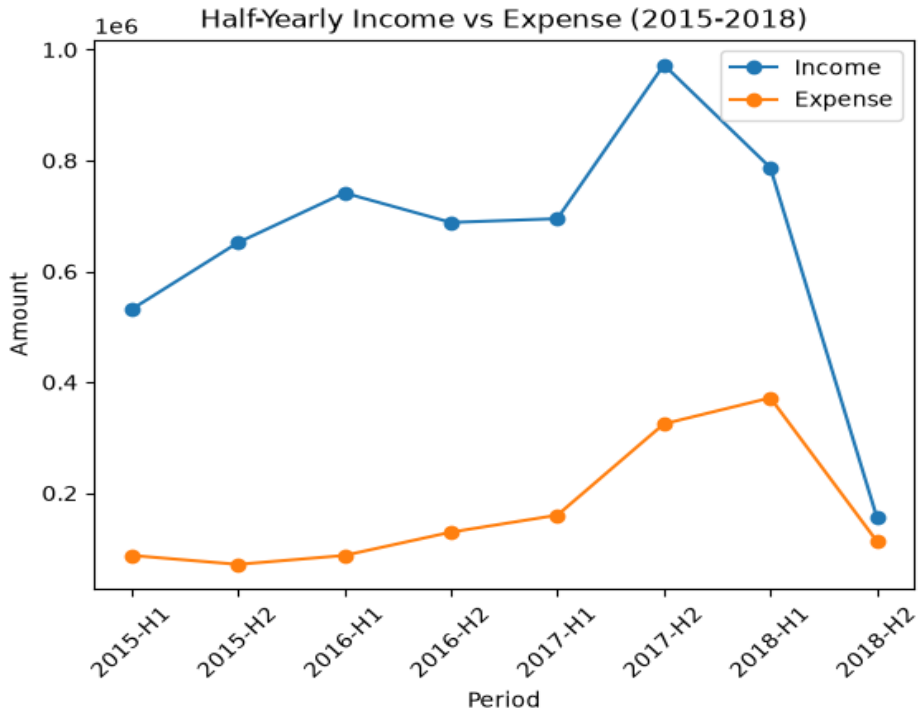


Interpretation: Transportation and household expenses account for the largest share of spending, followed by money transfers and subscriptions.

2. Monthly Income vs Expense

This pipeline groups documents by year, month and transaction type (`$year`, `$month`, `income_expense`) and sums the amounts. The results were sorted chronologically. Sample output:

```
Monthly_Income_vs_Expense=[{"$group":
                           {"_id":
                            {"year":{"$year":"$date"},
                             "month":{"$month":"$date"},
                             "type":"$income_expense"
                             },
                           "total":{"$sum":"$amount"}
                           }},{
                             "$sort": {
                                 "_id.year": 1,
                                 "_id.month": 1
                               }
                         }]
result2=list(collection.aggregate(Monthly_Income_vs_Expense))
print(result2)
```

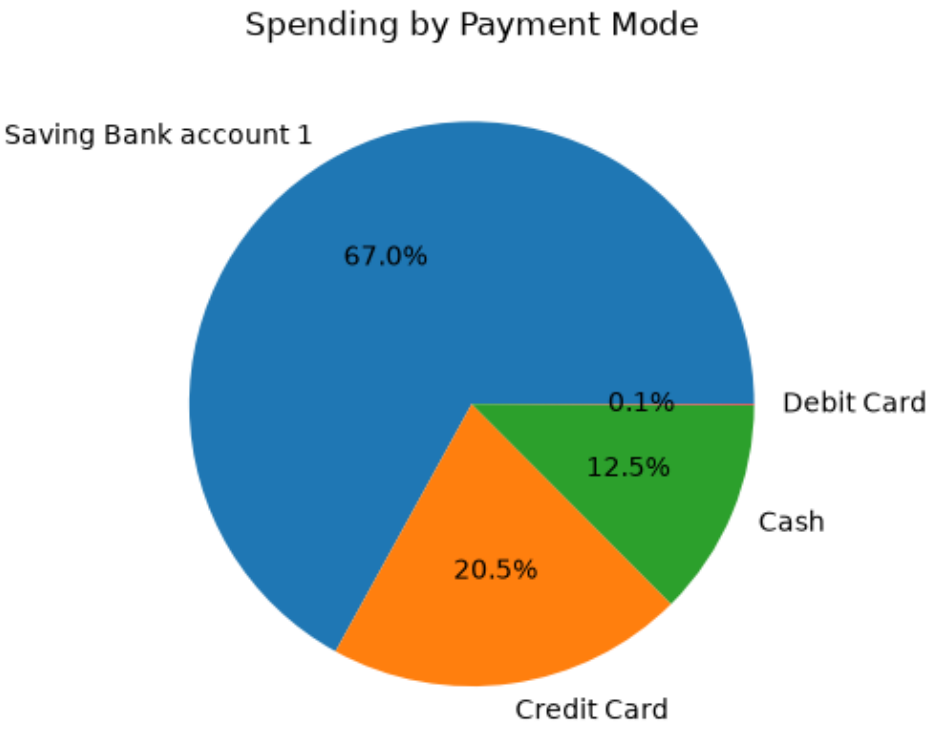


Interpretation: Income remained consistently higher than expenses in the sampled months, with expenses fluctuating more from month to month.

3. Spending by Payment mode

The pipeline groups expenses by payment mode and returns the four most-used modes.

```
spending_by_payment=[{"$match":{"income_expense":"Expense"}},
                     {"$group":{"_id":"$mode","total":{"$sum":"$amount"}}},
                     {"$sort":{"total":-1}},
                     {"$project":{"_id":0,"mode":"$_id","total":1}},
                     {"$limit":4}]
result3=list(collection.aggregate(spending_by_payment))
print(result3)
```

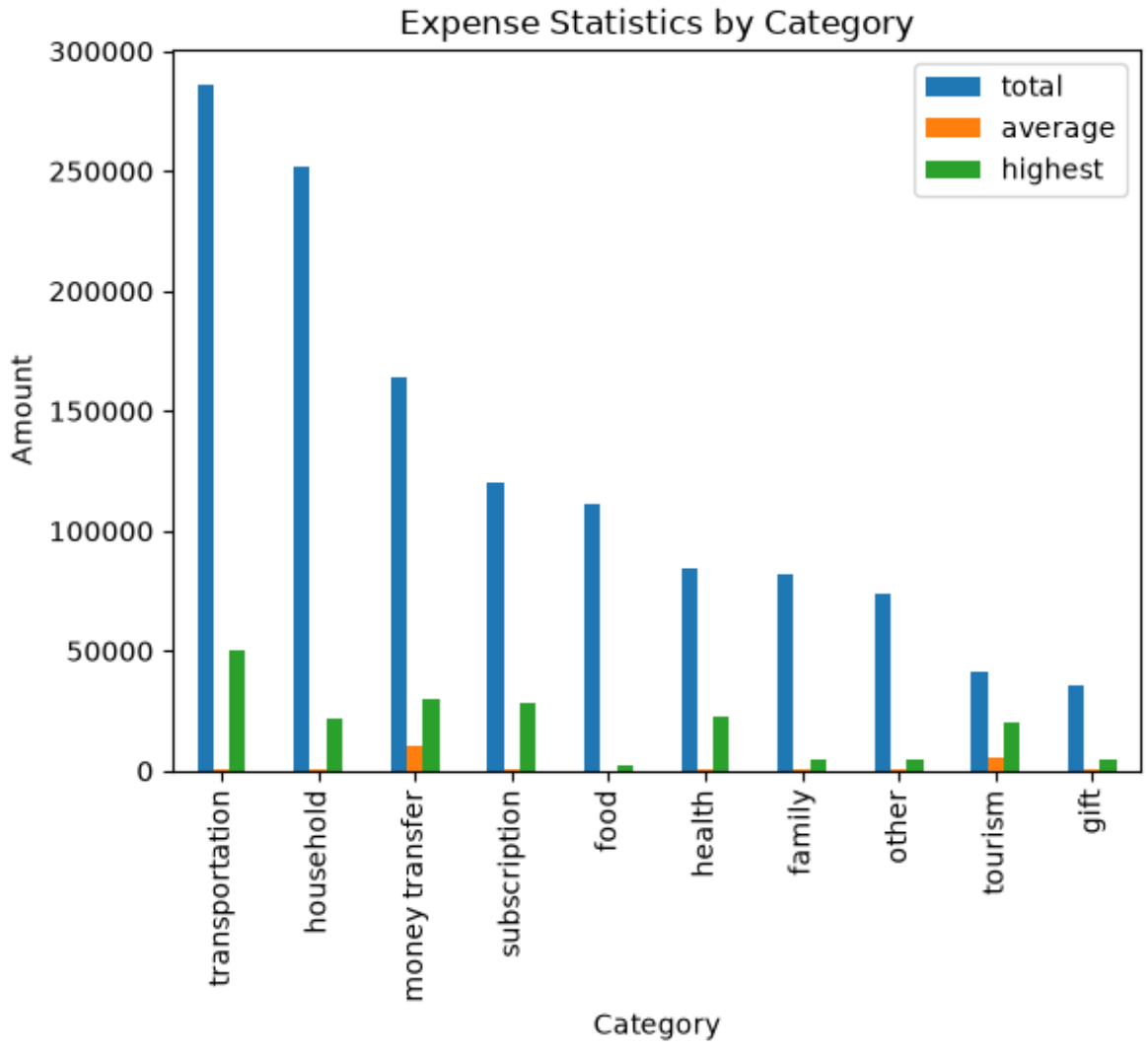


Interpretation: About two-thirds of all spending was made through the savings bank account, while debit cards were almost unused.

4. Category Statistics

This pipeline calculates the total (`$sum`), average (`$avg`) and highest (`$max`) amount for each expense category.

```
category_statistics=[{"$match":{"income_expense":"Expense"}},
                     {"$group":{"_id":"$category","total":{"$sum":"$amount"},"average":{"$avg":"$amount"},"highest":{"$max":"$amount"}}},
                     {"$sort":{"total":-1,"average":-1,"highest":-1}},
                     {"$limit":10}]
result4=list(collection.aggregate(category_statistics))
print(result4)
```

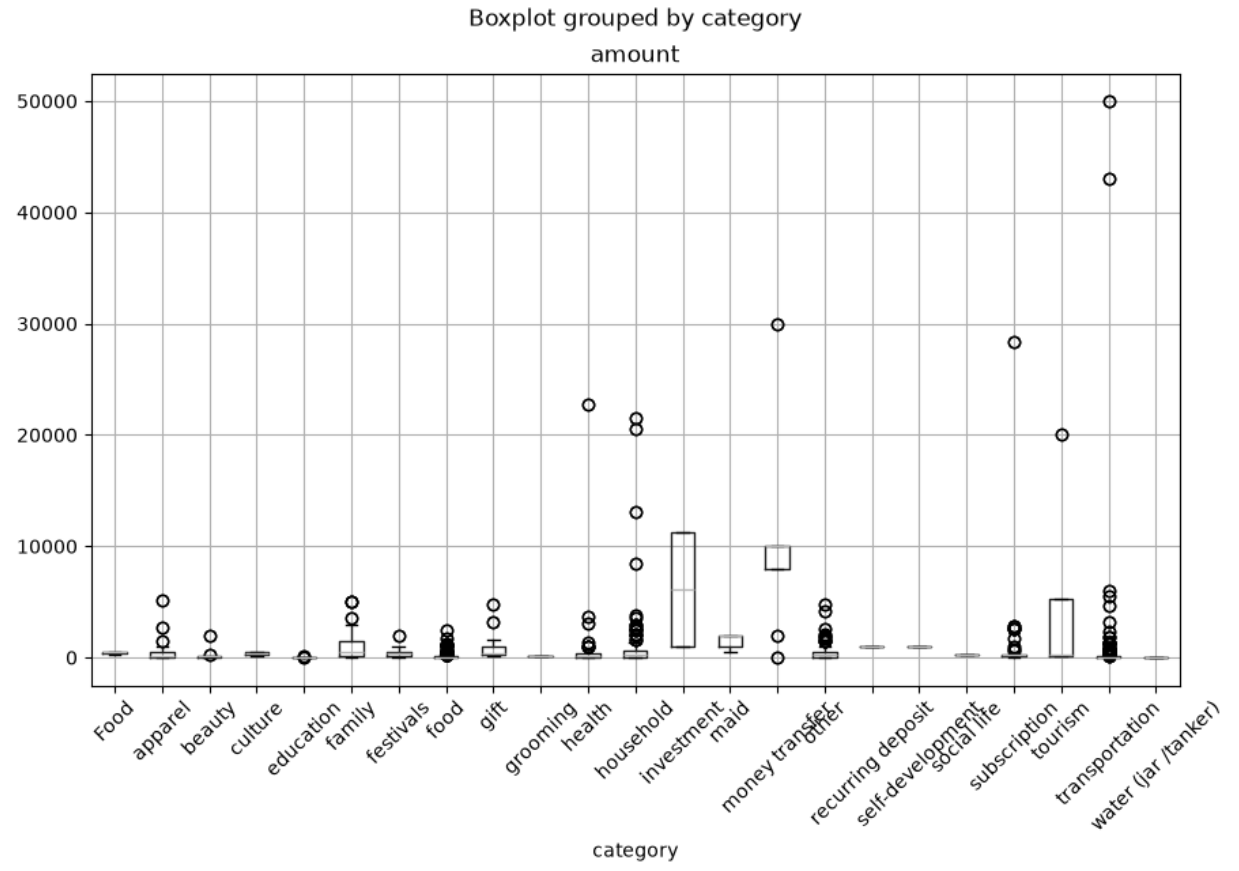


Interpretation: Food has a low average (about ₹108) because it consists of many small purchases, whereas money transfers and tourism have high averages because they consist of a few large payments.

5. Expense Distribution

A final pipeline projects only the `category` and `amount` fields of every expense. The resulting data was plotted as a box plot by category to show the spread of amounts and to highlight outliers such as the ₹50,000 transportation expense.

```
expense_distribution=[{"$match":{"income_expense":"Expense"}},
                      {"$project":{"_id":0,"category":1,"amount":1}}]
result5=list(collection.aggregate(expense_distribution))
print(result5)
```



AI-ASSISTED RECEIPT ENTRY

To reduce manual entry, the notebook includes an image-based workflow:

1. The user uploads a receipt image through an ipywidgets FileUpload control.
2. The image is sent to the Google Gemini API with a prompt instructing the model to return only a JSON object containing date, mode, category, subcategory, note, amount, income_expense and currency.
3. The JSON response is parsed with json.loads() and the date string is converted to a Python datetime object so that it matches the validator.
4. The document is stored using insert_one().

Sample:




```
Uploaded file: files/zvy1ypgnlbkc
{
    "date": "2019-12-13",
    "mode": "Cash",
    "category": "Food",
    "subcategory": "Restaurant",
    "note": "Domino's Pizza",
    "amount": 258.96,
    "income_expense": "Expense",
    "currency": "INR"
}
```

The document was inserted successfully (inserted ID 6ab41d375520301c50c6cba1). The Gemini API key is loaded from an environment variable using python-dotenv and is not hard-coded in the notebook.

```
data=json.loads(response.text)
data["date"]=datetime.strptime(data["date"],"%Y-%m-%d")
print(data)
```

```
{'date': datetime.datetime(2019, 12, 13, 0, 0), 'mode': 'Cash', 'category': 'Food',
```

```
result=db.transactions.insert_one(data)
print(result.inserted_id)
```

```
6ab41d375520301c50c6cba1
```

PYTHON-MONGODB INTEGRATION

The project uses PyMongo to connect Python to a local MongoDB server:



All database operations are executed from Jupyter Notebook cells. Aggregation results are returned as Python lists of dictionaries, which are converted into Pandas DataFrames for plotting. The notebook is organised into sections for data download and cleaning, database initialisation with validation, CRUD operations, receipt extraction, and aggregation with visualisation.

RESULTS AND DISCUSSION

The implemented application shows that MongoDB can store and manage personal financial transactions in a flexible document structure.

- **Data quality:** Cleaning reduced the raw dataset from 2,461 to 1,303 rows, and validation rules enforce correct types, a non-negative amount, and fixed values for transaction type and currency.
- **CRUD:** Insert, read and delete operations executed successfully. The update operation matched no document for the chosen filter and therefore modified none.
- **Spending patterns:** Transportation (₹286,132) and household (₹251,551.16) were the highest-spending categories. Food was the most frequent category but had a low average amount.
- **Payment behaviour:** Around 67% of spending was done through the savings bank account.
- **Income vs expense:** Aggregation by month and half-year allowed income and expenses to be compared over time.
- **Automation:** The Gemini-based receipt reader successfully converted a receipt image into a valid transaction document.

Limitations: The dataset is a single household's records, no unique index is defined on transaction_id, and AI-extracted values should be reviewed before being relied upon.

CONCLUSION

The MongoDB-Based Expense Tracker System demonstrates how MongoDB and PyMongo can be used to build a database application for a real-world problem. The project covers dataset preparation, document design, schema validation, CRUD operations, aggregation pipelines, data visualisation and AI-assisted data entry. The resulting system provides a foundation for more advanced expense analysis and personal budgeting applications.

FUTURE SCOPE

1. **Budget limits and alerts:** Monthly budgets per category with overspending notifications.
2. **Unique indexes:** A unique index on transaction_id to prevent duplicate records.
3. **User accounts:** Authentication and per-user transaction collections.
4. **Web or mobile interface:** A Flask/FastAPI backend with a dashboard front end.
5. **Forecasting:** Predictive models for future spending using Python or Power BI.
6. **Report export:** Export of summaries to CSV or PDF.
7. **Improved receipt reading:** Confirmation screen for AI-extracted data before saving.
8. **Cloud deployment:** Hosting the database on MongoDB Atlas for access from multiple devices.

REFERENCES

1. MongoDB Documentation — https://www.mongodb.com/docs/
2. PyMongo Documentation — https://pymongo.readthedocs.io/
3. MongoDB Manual: Schema Validation — https://www.mongodb.com/docs/manual/core/schema-validation/
4. MongoDB Manual: Aggregation — https://www.mongodb.com/docs/manual/aggregation/
5. Daily Household Transactions Dataset (Kaggle, prasad22) — https://www.kaggle.com/datasets/prasad22/daily-transactions-dataset
6. Pandas Documentation — https://pandas.pydata.org/docs/
7. Matplotlib Documentation — https://matplotlib.org/stable/
8. Google Gemini API Documentation — https://ai.google.dev/gemini-api/docs